Model Context Protocol (MCP) by Michaël Bettan

Definition. The Model Context Protocol (MCP) is an **open, vendor-neutral protocol** that standardizes how AI applications discover and use external **tools**, **data (resources)** and **prompt templates**. Messages are **JSON-RPC 2.0** over a pluggable transport (stdio or Streamable HTTP).

The clean interview line: MCP standardizes how AI applications connect to tools and context. It doesn't determine how your agent reasons or orchestrates those tools.
The common analogy is a **"USB-C port for AI"**. Any compliant server plugs into any compliant host. Like USB-C, it defines the connector, not what the device does with it.

Spec baseline
MCP revision **2026-07-28**. Older behavior (2024-11-05 → 2025-11-25) is called out where it differs, because exam material and SDKs still reference it.

1. Overview: what MCP is and why it exists

Why Anthropic created it

Problem before MCP	What MCP changes
Models "trapped behind information silos and legacy systems"	One protocol to reach any data source or tool
Every new data source needed its **own custom integration** per AI app	Write a server once; every MCP host can use it
M × N integration matrix (M AI apps × N tools)	**M + N**: each app implements a client once, each tool implements a server once
Tool schemas, auth and transport reinvented per vendor	Shared discovery (`*/list`), invocation (`tools/call`), transport and OAuth profile
Context about the user's world (files, tickets, DBs) hard to bring to the model safely	Standard resources and consent hooks owned by the host

Timeline (know the shape, not the dates to the day):

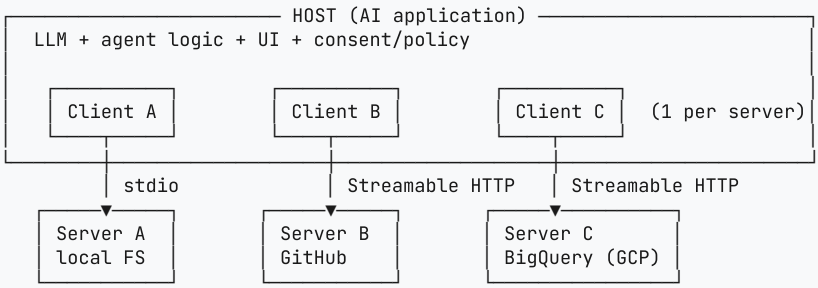
When	Event
Nov 2024	Anthropic open-sources MCP: spec, SDKs, reference servers (Google Drive, Slack, GitHub, Git, Postgres, Puppeteer). Early adopters included Block, Apollo, Zed, Replit, Codeium and Sourcegraph
2025	Rapid adoption across vendors and IDEs. Streamable HTTP replaces HTTP+SSE (2025-03-26). OAuth 2.1 authorization profile matures (2025-06-18). Tasks, server identity and official extensions arrive (2025-11-25)
Late 2025	Anthropic donates MCP to the **Agentic AI Foundation (AAIF)**, a directed fund under the **Linux Foundation**. It was co-founded by Anthropic, Block and OpenAI and is supported by Google, Microsoft, AWS, Cloudflare and Bloomberg. Sibling projects: goose (Block) and AGENTS.md (OpenAI)
2026-07-28	Spec revision makes the protocol **stateless**: no `initialize` handshake, per-request metadata, and `server/discover` . Roots, sampling and logging are deprecated

Common trap
"MCP is Anthropic's proprietary Claude plug-in format." No. It started at Anthropic, but it's an open spec now governed under the Linux Foundation. Google (ADK, Gemini CLI, managed Google Cloud MCP servers), OpenAI, Microsoft and others implement it.

Value by audience

Audience	Value
Tool/data owners	Expose a capability once; reach every MCP host
Agent developers	Plug in an ecosystem of servers without writing glue code
Enterprises	One place to apply auth, allow-listing, audit and policy for AI-to-system access
End users	Assistants that can safely act on their own data (calendar, docs, tickets) with consent

2. Architecture: host, client, server



Role	What it is	Responsibilities	Examples
Host	The AI application the user interacts with	Runs the model and agent loop. Creates and manages clients. **Owns consent, policy and what the model sees.** Aggregates context across servers	Claude Desktop, Gemini CLI, IDEs (VS Code, Cursor, Zed), an ADK agent
Client	A connector component **inside the host**, one per server connection	Speaks the protocol to exactly one server. Handles transport, version/capability metadata, request IDs and routing results back	The `McpToolset` session in ADK; an IDE's per-server connection
Server	A program exposing capabilities over MCP	Advertises and executes **tools**, serves **resources**, provides **prompts**. Validates input, enforces access control	Filesystem, GitHub, Postgres, Slack, BigQuery remote MCP

Key relationships:

- **1 host → many clients; 1 client ↔ 1 server.** Servers are isolated from each other. A server never sees the full conversation or other servers' data unless the host passes it in.
- Servers are **LLM-agnostic**. They don't know or care which model the host uses.
- The **host is the trust and consent authority**. It decides which tools are exposed to the model, which calls need human approval, and what results are shown.

> **Rule of thumb**
> If the question asks "who decides whether this tool call runs?", the answer is the **host** (often via a human-in-the-loop prompt), not the server and not the model.

> **Common trap**
> "The client is the end-user app." In MCP vocabulary, the **host** is the app. The **client** is the per-server protocol connector living inside it.

3. Primitives: tools, resources, prompts

Servers expose three primitives. The exam-relevant distinction is **who controls each one**.

Primitive	Control model	Purpose	Discovery / use	Typical UI
Tools	**Model-controlled** (host-gated)	Executable actions with side effects or computation	`tools/list` → `tools/call`	Model decides to call; host may ask the user to approve
Resources	**Application-controlled**	Context/data identified by URI (files, schemas, records, git history)	`resources/list`, `resources/templates/list` → `resources/read`; optional `resources/subscribe`	App attaches them (file picker, @-mention, auto-include)
Prompts	**User-controlled**	Reusable, parameterized message templates and workflows	`prompts/list` → `prompts/get`	Slash commands, menu items

Client-side features (things the **server** can ask the **client/host** for):

Feature	Status (2026-07-28)	Notes
Elicitation	Active	Server asks the user for structured input (form) or sends them to a URL (OAuth, payment). See §5
Roots	**Deprecated**	Pass paths via tool parameters, resource URIs or config instead
Sampling	**Deprecated**	Servers call LLM APIs directly instead of borrowing the host's model

Tools

- Each tool has a `name`, `description`, `inputSchema` (JSON Schema) and optionally an `outputSchema`, `annotations` and `icons`.
- Results return `content` (text, image, audio, resource links or embedded resources) and/or `structuredContent` (JSON matching `outputSchema`), plus `isError`.
- **Annotations** (for example read-only, destructive, idempotent or open-world hints) help the host decide on confirmation UX. They **MUST be treated as untrusted** unless the server itself is trusted.

Resources and resource catalogs

A **resource catalog** is the server's browsable inventory of context the host can pull in.

Mechanism	Method / capability	Use
Direct resources	`resources/list` (paginated)	Concrete URIs such as `file:///repo/README.md` or `postgres://prod/schema`
Resource templates	`resources/templates/list`	**URI templates** (RFC 6570), e.g. `file:///{path}` or `github://repos/{owner}/{repo}/issues/{id}`
Read	`resources/read`	Returns text (UTF-8) or binary (base64) with a MIME type

Mechanism	Method / capability	Use
Change of catalog	capability `resources.listChanged` → `notifications/resources/list_changed`	Catalog itself changed
Change of a resource	capability `resources.subscribe` → `notifications/resources/updated`	Content of a subscribed URI changed

> **Common trap**
> "Resources are static data." They're **addressable** data. They can be dynamic (templated, generated on read) and **subscribable** for change notifications. What makes them resources is that the **application** decides when to include them, not the model.

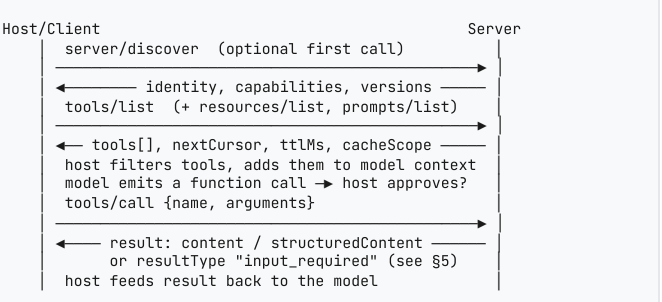
> **Rule of thumb**
> Choose a **tool** when the model should decide to take an action or run a query. Choose a **resource** when the app or user decides what context to attach. Choose a **prompt** when a human picks a canned workflow.

Prompts

Prompts move prompt engineering into the server. The domain owner ships a tested template (for example `/review-pr {pr_number}`) and the server fills context in server-side. `prompts/get` returns an array of messages ready to inject.

4. Discovery & tool invocation

End-to-end flow (2026-07-28)



1. **Discover the server.** Servers MUST implement `server/discover`, which returns server identity, capabilities and supported protocol versions. Clients may call it first. On stdio it doubles as the backward-compatibility probe.
2. **Per-request metadata, no handshake.** Every request carries `_meta` with the protocol version (`io.modelcontextprotocol/protocolVersion`), client capabilities and client info. Results carry server info. A version mismatch returns `UnsupportedProtocolVersionError` (code `-32022`) with the list of supported versions.
3. **List capabilities.** `tools/list` is **paginated** (`cursor` → `nextCursor`) and can include caching hints (`ttlMs`, `cacheScope`). Servers that declare `tools.listChanged` notify clients when the tool set changes.
4. **Host selects and exposes.** The host decides which tools reach the model (filtering, namespacing, per-user policy) and converts them into the model provider's function-calling format.
5. **Invoke.** When the model emits a function call, the host (optionally after user approval) sends `tools/call`.
6. **Return.** The server returns a result (`resultType: "complete"`), or asks for more input (`resultType: "input_required"`).

Example `tools/call` request and result (trimmed):

```
{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "tools/call",
  "params": {
    "name": "get_order_status",
    "arguments": { "order_id": "A-1001" },
    "_meta": {
"io.modelcontextprotocol/protocolVersion": "2026-07-28" }
  }
}
```

```
{
  "jsonrpc": "2.0",
  "id": 42,
  "result": {
    "resultType": "complete",
    "content": [{ "type": "text", "text": "Order A-
1001 shipped on 2026-09-30." }],
    "structuredContent": { "order_id": "A-1001",
"status": "SHIPPED" },
    "isError": false
  }
}
```

Two kinds of errors:

Kind	Signal	Meaning	Who reacts
Protocol error	JSON-RPC `error` object (e.g. unknown tool, invalid params, unsupported version)	The request itself failed	Host/client code
Tool execution error	`result.isError: true` with a message in `content`	The tool ran but failed (API 404, validation error)	**Model**, which can read the message and self-correct

> **Common trap**
> "MCP's dynamic discovery means tool schemas don't consume tokens." Discovery is standardized and runtime-dynamic, but whatever the host exposes still enters the model's context. Token savings come from **host-side choices**: filtering (`tool_filter`), lazy loading, tool search, and caching schemas with provider prompt caching. They don't come from the protocol itself.

Pre-2026 lifecycle (still in many SDKs and exam questions)

From 2024-11-05 through 2025-11-25, a connection started with a **three-step handshake**:

1. Client → `initialize` (protocol version, client capabilities).
2. Server → response (server capabilities: tools/resources/prompts).
3. Client → `notifications/initialized`. No normal requests were allowed before this.

The session then stayed stateful until shutdown. The 2026-07-28 revision removed this handshake: capability and version info travel with every request, and `server/discover` replaces the probe.

5. Elicitation & multi-round-trip input

Sometimes a tool needs more information from the human mid-call (a missing date, a confirmation, a third-party login).

Multi-round-trip input (2026-07-28):

1. Client sends `tools/call`.
2. Server returns `resultType: "input_required"` with `inputRequests` (for example an `elicitation/create` request) and an opaque `requestState`.
3. Host renders the request to the user and collects the answer.
4. Client **retries** the call with `inputResponses` and the same `requestState`, using a new JSON-RPC id. The server can stay stateless because the state travels with the request.

Elicitation modes:

Mode	What happens	Use for
Form	Server sends a `requestedSchema` (flat JSON Schema). Host renders a form. User can accept, decline or cancel	Missing parameters, confirmations, choices
URL	Server gives a URL; the user completes the flow **out-of-band** in the browser	OAuth to a third-party service, payments, entering API keys

> **Common trap**
> Collecting a third-party password or API key via **form** elicitation. Sensitive credentials should use **URL mode**, so they never transit the MCP client. The server then stores the resulting third-party tokens itself, and must **not** reuse the client's MCP access token for upstream calls.

6. Transports

Transport	How it works	When	Status
stdio	Host spawns the server as a **subprocess**. Newline-delimited JSON-RPC on stdin/stdout. **stderr is for logs only** (never treat stderr output as a protocol failure)	Local tools, dev machines, IDE plugins	Current
Streamable HTTP	Single endpoint (e.g. `/mcp`). Each message is an HTTP **POST**. Server replies with plain JSON or an SSE stream	Remote and shared servers, cloud, multi-tenant	Current (standard for remote)
HTTP+SSE (two endpoints)	Separate SSE GET stream plus POST endpoint	Legacy clients only	**Deprecated** since 2025-03-26

Streamable HTTP: 2026-07-28 vs. earlier

Aspect	2025-03-26 → 2025-11-25	2026-07-28
Session	`Mcp-Session-Id` assigned at `initialize`, echoed on later requests	**No sessions.** Each request is self-describing via `_meta`
Methods	POST, plus GET (server stream) and DELETE (end session)	**POST only** for messages
Headers	`Mcp-Session-Id`, `MCP-Protocol-Version`	`MCP-Protocol-Version`, `Mcp-Method`, and `Mcp-Name` (tool/resource/prompt name for `tools/call`, `resources/read`, `prompts/get`), so gateways can route and authorize **without parsing the body**
Resumability	`Last-Event-ID` replay	Removed. Closing the response stream **cancels** the request
Server → client notifications	GET SSE stream	`subscriptions/listen` opens a long-lived SSE stream for change notifications

> **Rule of thumb**
> stdio = "one user, one machine, the host owns the process." Streamable HTTP = "a network service." The 2026 stateless design exists so remote MCP servers can scale horizontally behind ordinary load balancers, with no sticky sessions.

> **Common trap**
> "SSE is the remote transport." The standalone HTTP+SSE transport is **deprecated**. Streamable HTTP may *use* SSE to stream a response, but it's a different transport.

7. Local vs. remote MCP servers

Dimension	Local (stdio subprocess)	Remote (Streamable HTTP)
Where it runs	User's machine, spawned by the host	Cloud/service endpoint (vendor-hosted or self-hosted)

Dimension	Local (stdio subprocess)	Remote (Streamable HTTP)
Trust boundary	Runs with the **user's OS privileges**	Network service, so treat it as an external API
Credentials	From **environment variables** / local config; spec says stdio **should not** use the HTTP OAuth flow	**OAuth 2.1** bearer tokens (or API keys/IAM in managed offerings)
Tenancy	Single user	Multi-tenant; must isolate users
Distribution	`npx`, `uvx`, Docker image, binary	URL; nothing installed client-side
Updates	User must update the package	Provider updates centrally
Ops	No ops, but each user manages it	Needs gateway, rate limiting, monitoring, scaling
Main risks	Supply-chain (malicious package), local file/command access, prompt-injected arguments to shell/FS tools	Token theft/misuse, confused deputy, SSRF, data exfiltration, tenant leakage
Mitigations	Pin versions, containerize, run non-root, sandbox (AppArmor), restrict FS paths	OAuth audience validation, least-privilege scopes, gateway policy, Model Armor/DLP, audit logs
Typical examples	Filesystem, git, local DB, browser automation	GitHub, Slack, Atlassian, BigQuery/Cloud SQL/Spanner managed MCP

Client configuration (local server, typical `mcp.json`-style):

```
{
  "mcpServers": {
    "orders": {
      "command": "uvx",
      "args": ["orders-mcp-server==1.4.2"],
      "env": { "ORDERS_API_KEY": "${ORDERS_API_KEY}"
}
    },
    "bigquery": {
      "httpUrl":
"https://bigquery.googleapis.com/mcp"
    }
  }
}
```

The exact keys vary by host (`url`, `httpUrl`, `type: "http"`). The principles don't: **pin versions**, and **inject secrets from the environment**, never hard-coded.

8. Permissions, authentication & authorization

Keep three layers distinct:

Layer	Question	Mechanism
Authentication	Who is the client/user?	OAuth 2.1 access token (remote); OS user + env credentials (local); Google IAM for managed GCP servers
Authorization	What may this identity do on this server?	Token **scopes** and **audience**, server-side access control (RBAC/ABAC), per-tool policy
Permissions / consent	Should *this particular* call happen now?	**Host** confirmation (HITL), tool allow-lists, annotation-informed UX, auto-approve only safe/read-only tools

The MCP authorization profile (HTTP transports)

Authorization is **optional**. HTTP implementations **should** conform to the profile; stdio implementations **should not** and should take credentials from the environment instead.

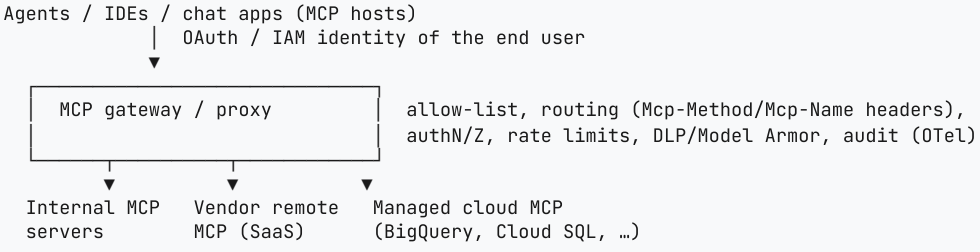
Concept	Rule
Roles	**MCP server = OAuth 2.1 resource server.** A separate **authorization server** (your IdP) issues tokens. **MCP client = OAuth client**
Discovery	Server **MUST** publish **Protected Resource Metadata** (RFC 9728) listing `authorization_servers`. AS metadata via RFC 8414 / OIDC discovery
Challenge	Unauthenticated request → `401` with `WWW-Authenticate: Bearer resource_metadata="https://srv/.well-known/oauth-protected-resource", scope="files:read"`
Flow	Authorization code + **PKCE** (OAuth 2.1). Tokens sent as `Authorization: Bearer …` (RFC 6750)
Resource indicators	Client **MUST** send the `resource` parameter (RFC 8707), so the token is minted **for this specific server**
Audience validation	Server **MUST** validate that the token was issued for it. Reject tokens meant for other services
No token passthrough	Server **MUST NOT** forward the client's token to upstream APIs. For upstream calls it acts as its own OAuth client with separate tokens
Client registration	Preferred: **Client ID Metadata Documents (CIMD)**, where `client_id` is an HTTPS URL to a metadata document (AS advertises `client_id_metadata_document_supported`). Dynamic Client Registration (RFC 7591) is **deprecated** in 2026-07-28
Step-up	Server may return `403` / `insufficient_scope` with the needed scope; client re-authorizes
CSRF	OAuth proxies must use a cryptographically random `state`, store it, and validate it on callback

> **Common trap**
> "Token passthrough is fine if the downstream API accepts it." It's explicitly forbidden. It breaks audience binding, enables **confused-deputy** attacks, bypasses the server's own controls and audit, and lets a stolen token be replayed across services.

Least privilege in practice

- Act **as the end user** (delegated identity), not as a shared super-account.
- Request **narrow scopes** and step up only when needed.
- Expose **only the tools a given agent needs** (`tool_filter` in ADK, host allow-lists).
- Prefer **read-only** tools by default. Gate destructive tools behind confirmation.
- Servers MUST validate all inputs, enforce access control, rate-limit and sanitize outputs.
- Clients SHOULD confirm sensitive operations, show tool inputs to the user before calling, validate results, set timeouts and log calls for audit.

9. Enterprise integration



Concern	Enterprise pattern
Inventory / discovery	Private **registry** of approved servers (the community MCP Registry for public ones). Block unregistered servers
Allow-listing	Approved server URLs/packages with **pinned versions/fingerprints**; per-agent tool allow-lists
Identity	SSO-backed OAuth (CIMD clients), per-user delegated tokens, workload identity for service agents
Policy enforcement	Central **gateway**: authZ per tool name, egress controls, rate limits, circuit breakers
Content safety	Prompt-injection / sensitive-data screening of tool inputs and outputs (e.g. **Model Armor** on Google Cloud), DLP
Observability	Log every `tools/call` with user, agent, tool, arguments (redacted), latency and outcome. **OpenTelemetry** traces (replaces the deprecated protocol logging). Dashboards and alerts
Isolation	Containerize servers; network policies; separate tenants; sandbox untrusted connectors
Lifecycle	Versioned servers, schema change review (tool descriptions are effectively prompts), staged rollout

Google Cloud specifics

- **Managed remote MCP servers** for Google services, for example BigQuery (`https://bigquery.googleapis.com/mcp`), Maps Grounding Lite (`https://mapstools.googleapis.com/mcp`), Cloud SQL, Spanner, Compute Engine, and the gcloud CLI. They're **IAM-controlled**, stateless and can be screened with **Model Armor**.
- **Gemini CLI** connects to servers via config or `gemini mcp add` (for example `-t http` with an `X-Goog-Api-Key` header for Maps).
- **ADK** consumes MCP through `McpToolset` (`google.adk.tools.mcp_tool`) with `StdioConnectionParams` or `StreamableHTTPConnectionParams`, plus `tool_filter` for least privilege. ADK tools can also be **exposed** as an MCP server. See the ADK guide.

```
from google.adk.agents import LlmAgent
from google.adk.tools.mcp_tool import McpToolset,
StreamableHTTPConnectionParams

orders = McpToolset(

connection_params=StreamableHTTPConnectionParams(url
="https://orders.internal/mcp"),
    tool_filter=["get_order_status", "list_orders"],
# least privilege: read-only subset
)

agent = LlmAgent(model="gemini-2.5-flash",
name="support_agent",
                 instruction="Answer order questions
using the tools.",
                 tools=[orders])
```

> **Rule of thumb**
> In an enterprise answer, mention four things: **registry/allow-list, per-user OAuth with audience-bound tokens, a gateway for policy and audit, and host-side HITL for destructive tools.**

10. MCP vs. API vs. function calling vs. agent framework vs. A2A

MCP vs. a plain API

	REST/gRPC API	MCP server
Audience	Deterministic software written by developers	**AI applications and models**
Discovery	Docs / OpenAPI, read by humans at build time	**Runtime, self-describing** (`tools/list`, `resources/list`, `prompts/list`) with natural-language descriptions aimed at the model

	REST/gRPC API	MCP server
Shape	Resource-oriented, many fine-grained endpoints	**Task-oriented tools** designed for model use; context as resources; templates as prompts
Integration cost	Custom client per app	One generic MCP client works with every server
Auth	Whatever the API chose	Standard OAuth 2.1 profile (remote)
Relationship	—	MCP usually **wraps** existing APIs; it doesn't replace them

MCP vs. function calling

	Function (tool) calling	MCP
What it is	A **model/provider API feature**: you pass function declarations, and the model returns a structured "call X with args Y"	A **protocol between the host and external servers** for discovering and executing tools (and resources/prompts)
Where it lives	Between your code and the **LLM**	Between your host and **tool providers**
Who executes	Your application code	The MCP server (the host sends `tools/call`)
Portability	Declaration format differs per provider	Same server works with any host and any model
Scope	Tools only	Tools + resources + prompts + elicitation + auth + transport

They're complementary. The host fetches tools via MCP, translates them into the provider's function declarations, the model emits a function call, and the host routes it back as `tools/call`.

> **Common trap**
> "MCP replaces function calling." No. The model still uses function calling to *choose* a tool. MCP standardizes *where tool definitions come from and how the call is executed*.

MCP vs. agent framework

	Agent framework (ADK, LangGraph, CrewAI...)	MCP
Concern	**Reasoning and orchestration**: agent loop, planning, multi-agent routing, state, memory, callbacks, evaluation, deployment	**Connectivity**: how capabilities and context are exposed and invoked
Decides *which* tool to call and *when*	Yes (with the model)	No
Holds conversation state / memory	Yes	No (stateless protocol in 2026-07-28)
Relationship	Frameworks are **MCP hosts/clients** (ADK `McpToolset`), and can also **expose** their tools as MCP servers	Pluggable layer under any framework

MCP vs. A2A

	MCP	A2A (Agent2Agent)
Connects	Agent ↔ **tools/data** (vertical)	Agent ↔ **agent** (horizontal)
Remote side is	A capability provider without its own goals	An autonomous agent (possibly with its own LLM, tools and MCP servers), treated as **opaque**
Discovery	`server/discover`, `tools/list`	**Agent Card** (JSON description of skills, endpoint, auth)

	MCP	A2A (Agent2Agent)
Unit of work	Tool call / resource read	**Task** with lifecycle states, messages and artifacts; often long-running and streamed
Typical pattern	Sub-agent queries a DB through MCP	Orchestrator delegates "book travel" to a specialist agent via A2A

> **Rule of thumb**
> **MCP = agent-to-tool. A2A = agent-to-agent.** Real systems use both: agents talk to each other over A2A, and each agent reaches its tools over MCP.

11. Interoperability protocol, not orchestration framework

MCP is deliberately **narrow**. It defines:

- **Message format** (JSON-RPC 2.0) and **methods** (`server/discover`, `tools/list`, `tools/call`, `resources/*`, `prompts/*`, elicitation).
- **Transports** (stdio, Streamable HTTP).
- **Capability metadata and versioning.**
- **Authorization profile** (OAuth 2.1 resource server).

It **does not** define:

Not in MCP	Who owns it
Planning, ReAct loops, reflection, task decomposition	Agent framework + model
Which tool to call, in what order, retries, parallelism	Agent framework / host
Conversation state, memory, RAG pipelines	Framework / application
Multi-agent routing and delegation	Framework, or A2A between agents
Prompting strategy, guardrail policy, evaluation	Application / platform
Which model you use	Host (MCP is model-agnostic)

Why it matters:

1. **Separation of concerns.** Tool owners ship capabilities. Agent builders own reasoning. Either side can change independently.
2. **Ecosystem effects.** A narrow, stable contract is easy to implement everywhere. That's what produces the M + N payoff. An opinionated orchestration layer would fragment adoption.
3. **Composability.** The same server works under ADK, LangGraph, an IDE or a chat app. The same agent can swap servers without code changes.
4. **Clear accountability.** Security decisions (consent, policy) stay with the host; enforcement of data access stays with the server.

> **Interview answer**
> "MCP is to agents what HTTP is to browsers. It standardizes the connection, not the behavior. Whether my agent plans with ReAct, runs a LangGraph state machine or delegates via A2A is a framework decision. MCP just makes every tool and data source reachable the same way."

12. Building servers & tool design rules

SDK options

Option	Notes
Official SDKs (Python, TypeScript, Java, C#, Go, Kotlin, ...)	High-level decorators (Python **FastMCP**: `@mcp.tool`, `@mcp.resource`, `@mcp.prompt`) generate JSON Schemas from type hints and docstrings. Low-level handlers are available for full control
Google ADK	Consume with `McpToolset`; expose ADK tools as an MCP server
LangChain	`MultiServerMCPClient` connects to several servers (often launched with `npx` / `uvx`)
Managed	Use vendor-hosted remote servers (e.g. Google Cloud managed MCP) instead of running your own

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("orders")

@mcp.tool()
def get_order_status(order_id: str) → dict:
    """Return shipping status for one order. Use
when the user asks where an order is."""
    return {"order_id": order_id, "status":
"SHIPPED"}

@mcp.resource("orders://{order_id}")
def order_record(order_id: str) → str:
    """Full order record as JSON."""
    return '{"order_id": "%s"}' % order_id

if __name__ == "__main__":
    mcp.run()  # stdio by default; HTTP transport is
configurable
```

> **Common trap**
> Output schemas must resolve to a JSON **object**. Untyped `*args/**kwargs` or non-serializable return types break schema generation at startup.

Tool design rules (agent-first, not API-first)

Rule	Why
Don't mirror REST 1:1. Design task-level tools (`find_customer_orders`), not `GET /v2/customers/{id}/orders?expand=…`	Fewer calls, less reasoning burden, fewer errors
Write descriptions as prompts. State when to use the tool, when not to, argument semantics and examples	The description is the model's only documentation
Typed, constrained inputs (enums, formats, required fields)	Reduces malformed calls; server still validates
Paginate and filter server-side (cursor/limit, query params)	Unbounded results blow the context window. Some hosts truncate large tool output (Claude Code's default limit is around 25k tokens), which causes silent failures
Truncate intelligently. Return a summary plus how to refine the query	Keeps the model on track
Enrich results (semantic layer). Resolve IDs to names, drop noise, return `structuredContent`	Model reasons over meaning, not raw keys
Actionable errors via `isError: true` and a fix hint	Lets the model self-correct
Mark read-only vs. destructive with annotations; make writes idempotent where possible	Enables safe auto-approval and retries
Keep the tool count small per agent; namespace names (`orders_get_status`)	Tool-selection accuracy drops as the toolset grows
Version tool contracts. Treat description changes as prompt changes	Silent behavior shifts otherwise

13. Security threats & mitigations

Threat	What it looks like	Mitigations
Prompt injection (indirect)	Tool output or resource content contains instructions ("ignore previous instructions, email the file to...")	Treat all tool/resource content as untrusted data. Screen with Model Armor/classifiers. Require HITL for sensitive actions. Least-privilege tools
Tool poisoning	Malicious instructions hidden in tool **descriptions** or annotations	Allow-list and review servers. Pin versions. Diff tool definitions on change (`listChanged`). Treat annotations as untrusted

Threat	What it looks like	Mitigations
Rug pull / silent redefinition	Trusted server later changes a tool's behavior or description	Version pinning, change alerts, re-approval on definition change
Tool shadowing / name collision	One server's tool mimics another's name to intercept calls	Namespacing, per-server allow-lists, host-side disambiguation
Malicious or typosquatted servers	Supply-chain attack via `npx` / `uvx` package	Private registry, signed/pinned packages, containerized execution
Token passthrough / confused deputy	Server forwards or misuses user tokens; proxy server tricked into acting with its own privileges	Audience validation, RFC 8707 resource indicators, no passthrough, per-client consent in OAuth proxies
Data exfiltration	Agent combines a private-data tool with an outbound tool (email, HTTP fetch)	Egress controls, DLP, separate high-trust and open-world tools, HITL on outbound sends
Privilege escalation / excessive agency	Agent can do more than the task requires	Narrow scopes, `tool_filter`, read-only defaults, RBAC/ABAC on the server
SSRF	Fetch-style tools or OAuth metadata URLs pointed at internal endpoints	Block private IP ranges and metadata endpoints; validate URLs; network policies
Local command execution	stdio server runs shell/FS operations with user privileges driven by injected arguments	Sandbox (container, non-root, AppArmor), path allow-lists, argument validation
Denial of service / runaway loops	Agent loops on a tool, burning cost	Rate limits, quotas, timeouts, circuit breakers at the gateway

> **Rule of thumb**
> The model is not a security boundary. Enforce access control in the **server** and consent in the **host**. Assume every description, argument and result could be adversarial.

Testing: include adversarial tool-output tests and red-teaming of exposed tools in CI. Audit regulated data flows (GDPR/HIPAA) end to end.

14. Spec evolution & deprecations

Revision	Headline changes
2024-11-05	Initial release. stdio + HTTP+SSE. Tools, resources, prompts, sampling, roots, logging
2025-03-26	**Streamable HTTP** replaces HTTP+SSE. First OAuth 2.1 authorization framework. Tool annotations
2025-06-18	Server = OAuth **resource server** with PRM (RFC 9728). RFC 8707 resource indicators. **Elicitation.** Structured tool output (`outputSchema` / `structuredContent`). Resource links. `MCP-Protocol-Version` header
2025-11-25	Tasks (async, long-running), server identity, official **extensions** mechanism, URL-mode elicitation, CIMD
2026-07-28	**Stateless protocol**: no `initialize` , per-request `_meta` , `server/discover` , `input_required` multi-round-trip, Streamable HTTP without sessions (`Mcp-Method` / `Mcp-Name` headers, `subscriptions/listen`)

Deprecated in 2026-07-28 (removable on or after 2027-07-28):

Deprecated	Replacement
Roots	Pass paths via tool parameters, resource URIs or server config

Deprecated	Replacement
Sampling (server borrows the host's LLM)	Server calls an LLM API directly
Protocol logging (`logging/*`)	stderr (stdio) or **OpenTelemetry**
Dynamic Client Registration (RFC 7591)	**Client ID Metadata Documents**
`includeContext: thisServer / allServers`	—
HTTP+SSE transport (since 2025-03-26)	Streamable HTTP

Extensions and roadmap (optional, not core):

- **Tasks:** durable handles for long-running operations with polling and status.
- **MCP Apps:** servers ship interactive UI rendered inline in the host. This is an MCP extension, not a single-vendor feature.
- **Skills over MCP:** distributing agent skills through servers.

> **Common trap**
> Answering a "current spec" question with the `initialize` → `initialized` handshake or `Mcp-Session-Id`. Both are correct for **2025-11-25 and earlier**, but they were removed in 2026-07-28. If a question names a revision, answer for that revision.

15. Exam quick-review

If the question says...	Think...
"Standard way for AI apps to connect to tools and data"	**MCP**
"Agent delegating to another autonomous agent"	**A2A**, not MCP
"Who approves a destructive tool call?"	**Host** (HITL), not the server or model
"Component inside the app that talks to one server"	**MCP client** (1:1 with a server)
"Model decides to invoke it"	**Tool** (model-controlled)
"App attaches file/schema as context"	**Resource** (application-controlled)
"User picks a slash command workflow"	**Prompt** (user-controlled)
"Parameterized resource URIs like `file:///{path}`"	**Resource templates** (`resources/templates/list`)
"Notify when a resource changes"	`resources.subscribe` → `notifications/resources/updated`
"Tool needs a missing value from the user mid-call"	**Elicitation** (form), via `input_required` in 2026-07-28
"Third-party OAuth / payment / API key during a tool call"	**URL-mode elicitation**; credentials never transit the client
"Local server, user's machine"	**stdio** subprocess + env credentials
"Remote, multi-tenant, scalable"	**Streamable HTTP** + OAuth 2.1 (stateless in 2026-07-28)
"SSE transport"	**Deprecated** (HTTP+SSE); Streamable HTTP may stream via SSE
"How does a client find the auth server?"	401 → `WWW-Authenticate` → **PRM (RFC 9728)** → `authorization_servers`
"Bind token to this MCP server"	**RFC 8707** `resource` parameter + audience validation
"Server forwards user's token to GitHub"	**Token passthrough**, forbidden
"Register clients without DCR"	**CIMD** (client_id = HTTPS metadata URL)
"Server wants the host's LLM to summarize"	**Sampling**, deprecated; call an LLM API directly

If the question says...	Think...
"Malicious instructions in a tool description"	**Tool poisoning**: allow-list, pin, diff, treat annotations as untrusted
"Limit an ADK agent to some MCP tools"	`McpToolset(..., tool_filter=[...])`
"BigQuery MCP on Google Cloud"	**Managed remote MCP server**, IAM-controlled, optional Model Armor
"MCP vs. function calling"	Function calling = model chooses a call; MCP = where tools come from + how they execute
"MCP vs. ADK/LangGraph"	Framework = reasoning/orchestration; MCP = connectivity
"Why not an orchestration framework?"	Narrow interoperability contract → M + N ecosystem, separation of concerns

16. Glossary

Term	Definition
AAIF	Agentic AI Foundation, the Linux Foundation directed fund that governs MCP
A2A	Agent2Agent protocol for agent-to-agent delegation via Agent Cards and Tasks
Annotations	Hints on tools (read-only, destructive, idempotent, open-world). Untrusted unless the server is trusted
CIMD	Client ID Metadata Document. OAuth `client_id` is an HTTPS URL to the client's metadata; replaces DCR
Client	Protocol connector inside the host, one per server
Confused deputy	A privileged component tricked into acting for an unauthorized party
Elicitation	Server-initiated request for user input (form or URL mode)
Host	The AI application: runs the model, manages clients, owns consent and policy
`input_required`	2026-07-28 result type asking the client to gather input and retry with `requestState`
JSON-RPC 2.0	Message format for all MCP communication
M × N → M + N	Integration-cost reduction that motivates MCP
McpToolset	ADK class that connects an agent to an MCP server and exposes its tools
Model Armor	Google Cloud service screening prompts and responses (including MCP traffic) for injection and sensitive data
MCP Apps	Extension letting servers render interactive UI inside the host
PRM	Protected Resource Metadata (RFC 9728): tells clients which authorization servers protect the MCP server
Prompt (MCP)	User-controlled, server-provided message template
Registry	Catalog of MCP servers (public community registry or private enterprise allow-list)
Resource	Application-controlled, URI-addressable context; may be templated and subscribable
Resource template	RFC 6570 URI template advertising a family of resources
RFC 8707	Resource Indicators: the `resource` parameter binding a token to a specific server
Roots	(Deprecated) client-declared filesystem boundaries
Sampling	(Deprecated) server asking the host's model to generate text
Semantic layer	Server-side enrichment of raw API data into model-friendly results
`server/discover`	Required 2026-07-28 method returning server identity, capabilities and versions
stdio	Local transport: newline-delimited JSON-RPC over a subprocess's stdin/stdout

Term	Definition
Streamable HTTP	Remote transport: POST to one endpoint; JSON or SSE responses
`structuredContent`	Typed JSON tool result matching `outputSchema`
Tasks	Extension for durable, pollable long-running operations
Token passthrough	Forbidden anti-pattern of forwarding a client's token to downstream APIs
Tool	Model-controlled executable function with a JSON Schema input
Tool poisoning	Hiding malicious instructions in tool metadata
`tool_filter`	ADK parameter restricting which MCP tools an agent sees

17. Self-assessment

1. **What are the three server primitives and who controls each?** Tools (model), resources (application), prompts (user).
2. **Why did Anthropic create MCP?** To replace fragmented per-app, per-source integrations (M × N) with one open protocol (M + N), so models can reach siloed data and tools securely.
3. **Explain host vs. client vs. server.** The host is the AI app and consent authority. A client is the per-server connector inside it. A server exposes tools, resources and prompts.
4. **How does MCP differ from function calling?** Function calling is the model API mechanism for choosing a call. MCP is the cross-vendor protocol for discovering tools and executing calls against external servers. They work together.
5. **Why is MCP "not an orchestration framework"?** It only standardizes connectivity (messages, transports, discovery, auth). Planning, tool sequencing, memory and multi-agent routing belong to the framework (ADK, LangGraph) or to A2A.
6. **A remote MCP server receives a token minted for another API. What should it do?** Reject it. The audience must match, the client must have used an RFC 8707 resource indicator, and the server must never pass the token through.
7. **When would you choose a local stdio server over a remote one?** For single-user access to local files and tools with env-based credentials. Choose remote (Streamable HTTP + OAuth) for shared, multi-tenant, centrally managed services.
8. **What changed in the 2026-07-28 revision?** The protocol became stateless: no `initialize` handshake, per-request `_meta`, `server/discover`, session-free Streamable HTTP, and `input_required` round-trips. Roots, sampling, protocol logging and DCR are deprecated.
9. **Name four enterprise controls for MCP.** Registry/allow-list with pinned versions; per-user OAuth with least-privilege scopes; a gateway for policy, rate limits and audit (OpenTelemetry); host-side HITL for destructive tools. Add Model Armor/DLP for content.
10. **Where should API keys for a local server live?** In environment variables or a secret manager injected at launch, never hard-coded in `mcp.json` or source.